

## Experiment 3: Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation

### Setting Up a Gradle Project

#### Step 1: Ensure Gradle is Installed

Before starting, verify that Gradle is installed on your Ubuntu system. Open a terminal and run:  
`gradle -v`

#### Step 2: Create a New Gradle Project

Gradle offers an interactive command to generate a new project using an initialization script. To create a basic project:

##### 1. Create a new directory for your project and navigate to it:

```
>mkdir gradleworkspace  
>cd gradleworkspace
```

##### 2. Run the Gradle `init` command:

```
>gradle init
```

You will be prompted to choose between a few options, you can either choose defaults or specify details as needed:

- Select type of build to generate: 1(Application)
- Select implementation language:1(Java)
- Enter target Java version: 21 (Or whichever Java you are having)
- Project name (default: `gradleworkspace`):
- Select application structure (default: Single application project):
- Select build script DSL: 2 (Groovy)
- Select test framework: JUnit 4
- Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no]

## Step 3: Explore the Project Structure

### Step 3: Understanding Gradle Build Scripts

#### A. Groovy DSL Example (build.gradle)

Below is an example of a basic build.gradle for a Java application using Groovy DSL:

```
plugins {  
    // Apply the Java plugin for compiling Java code  
    id 'java'  
    // Apply the application plugin to add support for building an application  
    id 'application'  
}  
  
group = 'com.example'  
version = '1.0-SNAPSHOT'  
  
repositories {  
    // Use Maven Central for resolving dependencies.  
    mavenCentral()  
}  
  
dependencies {  
    // Define your dependencies. For example, JUnit for testing:  
    testImplementation 'junit:junit:4.13.2'  
}  
  
application {  
    // Define the main class for the application.  
    mainClass = 'org.example.App'  
}  
  
// A custom task example: printing a greeting  
task hello {  
    doLast {  
        println 'Hello, Gradle!'  
    }  
}
```

## Step 4: Edit the settings.gradle (Optional)

1. Go to your Project folder (eg: gradleworkspace)
2. Vim settings.gradle
3. Comment or remove below part:

```
plugins {  
    // Apply the foojay-resolver plugin to allow automatic download of JDKs  
    id 'org.gradle.toolchains.foojay-resolver-convention' version '1.0.0'  
}
```

## Step 5: Running and Verifying Gradle Project

### 1. Build the Project:

>gradle build

#### What it does:

- o Compiles source code (compileJava).
- o Runs tests (test).
- o Packages the application into a JAR file (jar)

### 2. Run the Application:

> gradle run

#### What it does:

Runs your main class as specified in the application block.

### 3. Executing Custom Tasks

> gradle hello

#### Expected Output:

The custom greeting message you defined earlier.

## A. Kotlin DSL Example (build.gradle.kts)

Here's how the same build configuration might look in Kotlin DSL:

### 1. Rename build.gradle to build.gradle.kts

mv build.gradle build.gradle.kts

### 2. Add the following content to build.gradle.kts:

```

plugins {
    java
    application
}
group = "com.example"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}

dependencies {
    testImplementation("junit:junit:4.13.2")
    implementation("org.apache.commons:commons-lang3:3.12.0")
}

application {
    mainClass.set("org.example.App")
}

tasks.test {
    useJUnit()
}

tasks.register("hello") {
    doLast {
        println("Hello, Gradle with Kotlin DSL!")
    }
}

```

**3. Rename settings.gradle to settings.gradle.kts:**

mv settings.gradle to settings.gradle.kts

**4. Edit the settings.gradle.kts**

vim settings.gradle.kts

```

rootProject.name = "gradleworkspace"
include("app")

```

**5. Build and run the project using the same steps mentioned above.**